

Search Typeahead System.

A search-autocomplete backend built around one idea: keep the read path off the database entirely. A prefix trie answers every keystroke in memory, a hand-written consistent-hash ring spreads cached results across three real Redis nodes, recency-aware ranking surfaces what is trending now, and writes are batched so popularity counts never touch SQLite one search at a time.

Stack: Node.js · Express · node:sqlite · Redis ×3 (Docker) · **Dataset:** 149,803 Wikipedia page titles · **Ring:** 150 virtual nodes/node

9.14 ms

P95 LATENCY · REDIS

1

DB READ / 20K REQUESTS

99.4 %

CACHE HIT RATE

~18_x

WRITE REDUCTION

01 Architecture

The system deliberately splits **reads** (suggestions) from **writes** (search submissions). Reads are served from memory and the cache and never block on the database. Writes are buffered

and applied to SQLite in aggregated batches. The trie and cache are derived, rebuildable views; SQLite is the single source of truth.

Browser UI

debounced GET /suggest · POST /search on enter

↓ GET /suggest

POST /search ↓

READ PATH · SUGGESTSERVICE

1. normalize prefix
2. consistent-hash → owning cache node
3. cache hit? return cached top-10
4. miss → walk trie for candidates
5. trending mode? recency re-rank
6. store in owning node (TTL), return

DistributedCache

ring →
redis cache-0/1/2
TTL per entry

Trie

in-memory
per-node top-K
 $O(\text{prefix len})$

WRITE PATH · BATCHWRITER

1. enqueue(query) → in-mem buffer
2. aggregate duplicates
3. flush every 2s **or** 500 distinct:
4. one SQLite transaction (UPSERT)
5. bump trie + record trending
6. invalidate touched prefixes

In-memory buffer

query → pending count
(lost on crash: ≤ 1 flush)

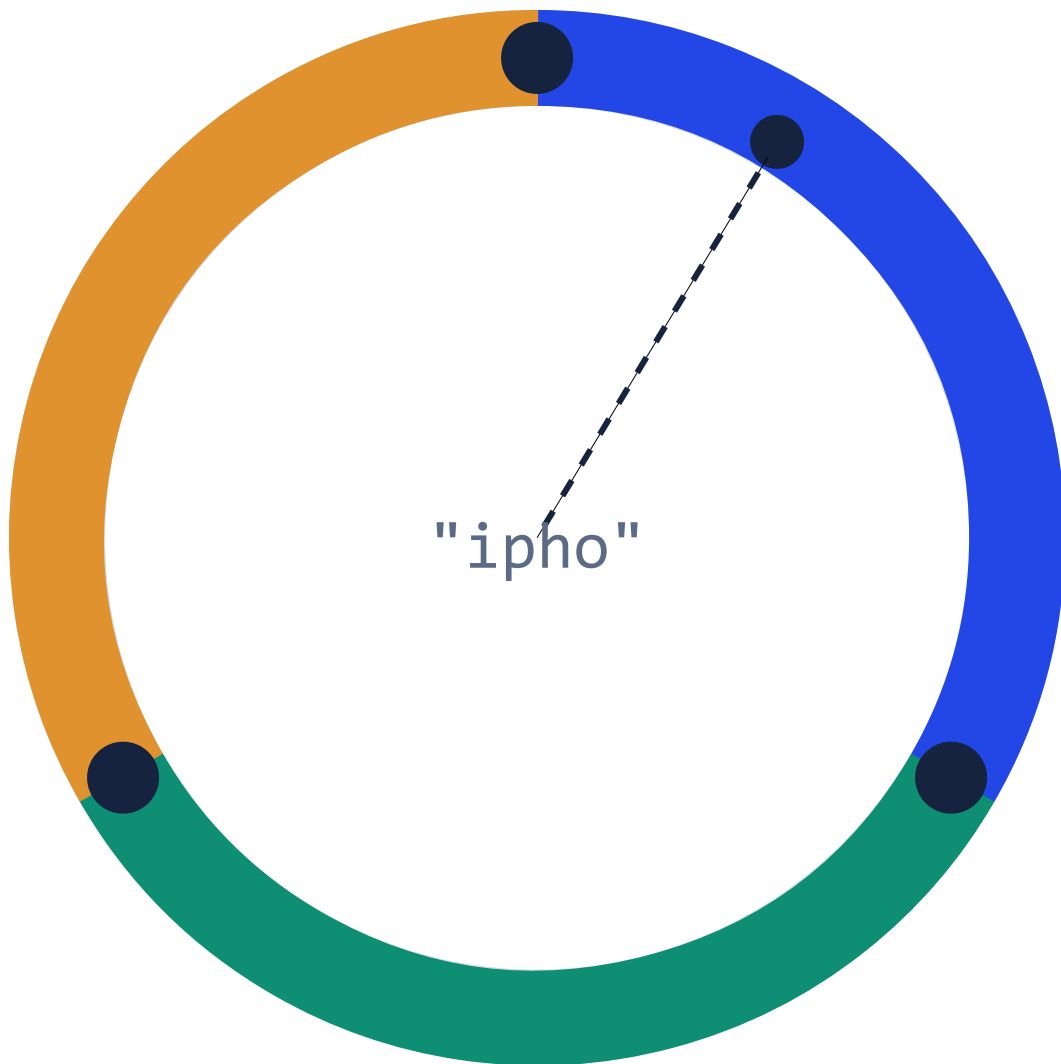
SQLite — queries(query, count, last_seen)

source of truth · trie built from here at startup · only the batch flush writes

Components

MODULE	ROLE
<code>server.js</code>	Express app and routes; wires components together.
<code>suggestService.js</code>	Read path: cache → trie → optional recency re-rank.
<code>trie.js</code>	Prefix trie; each node caches the top-K completions in its subtree.
<code>consistentHash.js</code>	Hash ring with virtual nodes; maps a prefix to one cache node.
<code>cache.js</code>	Distributed cache over the ring; Redis or in-memory backend, TTL per entry.
<code>trending.js</code>	Exponentially-decaying recency score per query.
<code>batchWriter.js</code>	Buffers searches, aggregates duplicates, flushes to SQLite in batches.
<code>db.js</code>	SQLite primary store (node:sqlite); single batched write path.

Consistent-hash routing



Each node owns an arc of the ring. A prefix is hashed to a point and walks **clockwise** to the first node it meets — so "ipho" lands on cache-0. Each physical node holds **150 virtual positions** on the ring, which keeps the load even and means adding or removing a node only remaps about **1/N** of keys instead of nearly all of them.

● cache-0 ● cache-1 ● cache-2

Why reads never hit SQLite

After the one-time startup load, suggestions come from the cache or the trie — both in memory. SQLite is touched only by the startup load and by batched writes. A 20,000-request benchmark produced exactly **one** database read. That is the entire purpose of the trie + cache layer.

FAULT TOLERANCE

The cache is an optimization, not a hard dependency. If a Redis node is unreachable, the read degrades to a cache miss and is served from the trie instead of failing. Killing a node mid-traffic was verified to keep every request returning 200, with the cache resuming cleanly on recovery.

02 Dataset

Requirement: at least 100,000 queries, each with a count. **Source:** Wikipedia pageviews hourly dumps. Each line is `domain page-title view-count bytes` ; filtering English traffic for a single hour yields hundreds of thousands of `title → count` rows. The **page title becomes the query** and the **view count is the popularity signal**.

Why this dataset: it is fully open (no privacy issues, unlike the withdrawn AOL query log), it carries a genuine count per entry, it is reproducible from a fixed URL, and one hour of English traffic comfortably clears the 100k minimum. The bundled `data/queries.csv` holds **149,803 distinct queries** built this way.

Loading instructions

```
# install
npm install

# A – use the dataset committed in the repo (149,803 rows)
npm run load

# B – rebuild from a fresh Wikipedia dump, then load
node scripts/download-dataset.js 150000 20240115 12 # topN, YYYYMMDD, HH
npm run load

# C – tiny bundled sample (~116 rows), for a quick smoke test
npm run load:sample
```

`download-dataset.js` streams the `.gz`, keeps English titles, cleans them (underscores to spaces, drops namespace pages like `Special:` / `Talk:`, drops one-off noise), takes the top N by views, and writes `query,count`. `load.js` normalizes each query and UPSERTs it into SQLite; the trie is built from that table at startup.

VERIFIED END TO END

Run against a live dump: 7.75M lines scanned → 456K candidate titles → clean query, count output — counts sorted descending, zero namespace junk, zero malformed rows.

03 API documentation

Base URL `http://localhost:3000`. All inputs are normalized (trimmed, lowercased).

Endpoints fail soft — bad input returns a 4xx with a clear body, never a stack trace.

GET `/suggest?q=<prefix>`

Up to 10 prefix matches, ranked by count (or recency-aware score in trending mode).
Empty/missing q returns the trending list; an unknown prefix returns an empty list with HTTP 200.

```
{
  "prefix": "ip",
  "suggestions": [
    { "query": "iphone", "count": 100000, "score": 11.51 },
    { "query": "ipad", "count": 67000, "score": 11.11 }
  ],
  "cached": false, "nodeId": "cache-1", "source": "trie"
}
```

POST `/search`

Body { "query": "<text>" }. Buffers the search for the next batch flush — does not write synchronously — and returns the dummy response. Missing/blank query → 400; malformed JSON → {"error": "invalid JSON body"}.

```
{ "message": "Searched" }
```

GET `/trending?limit=<n>`

Current trending queries, ordered by **decayed recent activity** (not raw count). Falls back to all-time top when nothing recent exists, so the panel is never empty.

```
{ "trending": [ { "query": "iphone", "count": 100000, "recentWeight": 40.85 } ],
  "rankMode": "trending" }
```

GET /cache/debug?prefix=<p>

Which cache node owns a prefix and whether it is currently cached — demonstrates consistent-hash routing. This call peeks; it does not count as a hit or miss.

```
{ "prefix": "ip", "node": "cache-1", "hit": true, "ttlRemainingMs": 29759 }
```

GET /stats

Live counters for the performance report: cache backend / nodes / hit rate / keys-per-node, DB read & write counts, and batch buffer depth + write-reduction ratio.

```
{ "cache": { "backend":"redis", "hitRate":0.9944,
             "keysPerNode":{"cache-0":9,"cache-1":2,"cache-2":8} },
  "db": { "reads":1, "writes":2 },
  "batch": { "enqueued":90, "rowsWritten":5, "writeReduction":18.0 } }
```

GET /health

Liveness probe.

```
{ "ok": true }
```

04 Design choices & trade-offs

Stack

LAYER	CHOICE	REASON
Backend	Express	One language across the stack; the trie/cache/ring logic reads cleanly in JS.
Datastore	node:sqlite	One file, real SQL, no native module to compile; synchronous API makes the read/write paths easy to reason about and to count.
Cache	Redis ×3	Three independent processes (ports 6379–6381); the app routes across them via a ring it implements itself. An in-memory backend mirrors the same ring for tests/offline runs.
Frontend	Vanilla JS	No build step; a debounced input and a dropdown are all the UI needs.

Trie for suggestions

A prefix trie gives $O(\text{prefix length})$ lookups, independent of dataset size, because every node caches the top-K completions in its subtree. The alternative — a sorted array with binary search — finds the prefix range in $O(\log n)$ but must then scan and sort that whole range per request, which is far worse for short, hot prefixes like "a" that match a huge slice of the data. A key invariant keeps it cheap: counts only ever increase, so top-K is updated incrementally on each write rather than recomputing a subtree.

TRADE-OFF

The materialized top-K is memory-hungry (~679 MB RSS for 149,803 queries — see §5). That is the deliberate price of flat read latency, and it is tunable by lowering the candidate-pool size K.

Consistent hashing in application code

The ring lives in our code, **not** in a Redis cluster. A cluster would hash keys for us inside the client, hiding the exact mechanism the assignment asks us to implement. So we run three plain, independent Redis nodes and route ourselves: hash the prefix, walk the ring clockwise, pick the node. 150 virtual nodes per physical node plus a murmur3-style finalizer keep the load even — without virtual nodes a 3-node ring splits traffic very unequally.

WHY IT MATTERS

Adding or removing a node remaps only $\sim 1/N$ of keys versus nearly all of them for naïve $\text{hash}(\text{key}) \% N$. Measured: adding a 4th node moved 26.7% of keys, where modulo would move about 75%.

Cache-aside with TTL

`/suggest` is cache-aside: check the cache, compute from the trie on a miss, write back under a short TTL (default 30s). The TTL bounds staleness, and writes additionally invalidate the affected prefixes on flush.

TRADE-OFF

Suggestions can be up to one flush interval ($\sim 2s$) or one TTL stale — fine for typeahead, where freshness within seconds is plenty and read latency stays flat.

Batch writes

Writing to SQLite once per search would make the DB the bottleneck. Instead searches are buffered and duplicates aggregated, so 50 searches for one query become a single `count += 50` update.

FAILURE MODEL

The buffer is in memory, so a crash loses at most one flush interval of searches — and only popularity counts, never user data. The trie is rebuilt from SQLite (the source of truth) on restart. Hardening would mean a write-ahead log before acking, reintroducing the synchronous write batching exists to avoid.

Trending: continuous exponential decay

Rather than discrete hourly buckets, each query keeps a single exponentially-decaying recency accumulator with a ~ 10 -minute half-life. The suggestion score is `$\log(1 + \text{count}) + w$`

• `decayed_recent_weight` . A short spike decays to zero on its own, so it cannot permanently over-rank a query; decayed entries are pruned on each flush.

Availability over strict consistency

A cache-node failure degrades to a trie read rather than an error. We chose to keep suggestions working from the trie over surfacing a cache outage to the user — the right call for a best-effort suggestion feature.

05 Performance report

All figures are measured and reproducible with `npm run bench`. Environment: Node.js v24, Windows 11, single machine; 149,803-query dataset; 3 cache nodes, 150 virtual nodes each, 30s TTL; trending ranking; single-threaded sequential benchmark client (so latencies are per-request service time, not contended throughput).

9.14 ms

P95 • REDIS BACKEND

1.50 ms

P95 • MEMORY BACKEND

99.4 %

CACHE HIT RATE

Suggestion latency • GET /suggest

20,000 requests over a realistic prefix mix (60% hot, 30% warm, 10% cold), full dataset, on each cache backend:

METRIC	REDIS (3 CONTAINERS)	MEMORY (IN-PROCESS)
average	5.18 ms	0.55 ms
p50	4.56 ms	0.37 ms
p95	9.14 ms	1.50 ms
p99	15.2 ms	2.67 ms
throughput	~193 req/s	~1,800 req/s

The gap is the cost of being genuinely distributed: every Redis hit is a real TCP round-trip to a separate process; the memory backend is a same-heap Map lookup. Latency does not grow with dataset size – the trie makes a lookup $O(\text{prefix length})$.

Consistent-hash distribution & remap

100,000 keys across 3 nodes — a property of the ring, not the data:

NODE	KEYS	SHARE
cache-0	30,987	31.0%
cache-1	34,841	34.8%
cache-2	34,172	34.2%

Adding a 4th node remapped only 26.7% of keys (ideal for 3→4 is 25%); naïve hash % N would remap ~75%. That ~3× reduction in cache churn on a topology change is the entire reason consistent hashing is used.

Reads avoided, writes reduced, memory cost

- **Database reads:** 1 total across the 20,000-request run (the startup load). The read path never touches SQL.
- **Write reduction:** ~18× measured on repeated popular queries; the ratio scales with how repetitive and bursty traffic is (a controlled 60-search / 2-query test showed 30×).
- **Memory footprint:** ~679 MB RSS with the full dataset loaded — the cost of the materialized per-node top-K that buys $O(\text{prefix})$ reads, tunable via the candidate-pool size K.

Correctness & robustness testing

12/12

UNIT TESTS

56/56

API EDGE ×2 BACKENDS

24/24

UI (REAL BROWSER)

15/15

BATCH PATHS

pass

NODE-FAILURE DRILL

Coverage included empty/missing/whitespace/unicode/SQL-injection inputs, no-match, case-folding, malformed JSON, 60 concurrent requests; debounce coalescing, keyboard navigation, and error states in a real browser; the `FLUSH_SIZE` early-flush, aggregation, and shutdown-drain paths; and a Redis node killed mid-traffic with every request staying 200 by degrading to the trie.

Reproduce

```
npm run load                # full dataset
docker compose up -d        # 3 Redis cache nodes
CACHE_BACKEND=redis npm start # terminal 1 (omit env for memory backend)
npm run bench 20000         # terminal 2 -> latency + cache numbers
docker exec typeahead-cache-0 redis-cli dbsize # keys living in a node
```